

WEEKLY PROGRESS REPORT

Week 3: DNS Tunneling - Lab Deployment, AI Enhancement & Live Testing

Lê Quý Tôn

April 19, 2026

Project: CyberSec Internship - Cygnus-Security
Related Issues: #22

1. Contribution Scope

This report covers the practical deployment and live testing phases of the Intrusion Detection System (IDS) project. It involves transitioning the theoretical ID3 Decision Tree model from a static environment into a live, real-time Deep Packet Inspection (DPI) testing environment, validating the system against simulated real-world background traffic.

2. Objectives of This Week

- Upgrade the AI's feature extraction logic to drastically reduce False Positives using advanced heuristics.
- Integrate the Python engine with Suricata's real-time packet logs (`eve.json`) on the LAN interface.
- Implement and validate active network mitigation utilizing dynamic OS-level `iptables` rules.
- Aggressively stress-test the system using an automated Bash script that simulates both benign business traffic and chaotic malware generation.

3. Detailed Work

3.1 Description

Transitioned the detection engine into a live lab that enforces strict Layer 3 segmentation. The Python DPI engine was engineered to act as a continuous listener, parsing Suricata logs in real-time to classify traffic, actively block malicious nodes, and allow clean background noise to pass seamlessly.

3.2 Design / Method

AI Model and Feature Engineering Advancements:

To fortify the mathematical boundary and reduce False Positives on human-readable domains, the feature vector was expanded to four dimensions: [Subdomain.Length, Entropy, Digit.Ratio, Root.Domain.Length].

- **Digit-to-Letter Ratio:** Uses Regular Expressions (`re.findall(r'\d')`) to calculate the density of numbers, easily distinguishing Base32/64 encoded payloads from normal strings.
- **Root Domain Tracking:** Integrates the `tldextract` library to explicitly isolate and measure the length of the Root Domain, helping the AI detect suspicious or randomly generated Top-Level Domains (TLDs/DGAs).

Active Mitigation and the "Whitelist" DevSecOps Logic:

Designed a dynamic `iptables` injection mechanism. To ensure complete isolation, packets are dropped on both the `FORWARD` and `INPUT` chains. A hardcoded `GATEWAY_IP` Whitelist was embedded to prevent "Self-Lockout" during bidirectional traffic analysis.

3.3 Implementation

- Configured the Python engine as a continuous tailing listener (`f.seek(0, os.SEEK_END)`) for real-time interception.
- Patched the JSON parser to navigate Suricata 8.0's nested array structures (`dns_info["queries"]`) ensuring 100% accurate extraction of domain records.
- Deployed `traffic_simulator.sh` to generate 100 concurrent DNS requests: 50 legitimate domains (e.g., *vnexpress.net*) and 50 highly chaotic DGA strings generated autonomously via Linux `/dev/urandom`.

4. Results / Output

- **Real-time Detection:** The Python engine effortlessly filtered the 50 clean domains while instantly triggering [MALWARE ALERT] against all 50 generated DGA strings with near-zero latency.
- **Live Mitigation:** The engine accurately dropped traffic from infected IPs (Attacker/Victim) without disrupting the simulated clean traffic or blocking the Gateway itself.

5. Analysis

The live simulation proved the viability of automated defense mechanisms. The integration of `Digit.Ratio` effectively eliminated False Positives on long, meaningful Vietnamese domains. However, the empirical test revealed a highly valuable False Positive edge-case: the AI erroneously flagged Cloudflare CDN domains (e.g., `chat.openai.com.cdn.cloudflare.net`) as malware. Because Content Delivery Networks (CDNs) utilize exceptionally long, high-entropy CNAME records for load balancing, their mathematical signature closely mirrors that of DNS Tunneling exfiltration. This perfectly dictates the direction for Future Work: integrating massive real-world datasets to refine the AI's decision boundary.

6. Issues & Fixes

- **Issue:** Suricata 8.0 JSON structure changes caused empty domain logs (Entropy = 0.0).
- **Fix:** Updated the Python parser to dynamically extract `rrname` from nested JSON arrays.
- **Issue:** Gateway Self-Lockout (Firewall blocked its own IP upon analyzing outbound response packets).
- **Fix:** Implemented a hardcoded `GATEWAY_IP` Whitelist in the mitigation function to prevent self-blocking.
- **Issue:** Network choking (Packet Loss) due to heavy DPI processing.
- **Fix:** Disabled OS-level sleeping states, disabled vNIC Hardware Offloading, and optimized script loops with `sleep` intervals.

7. Reusable Content ★

(Active Mitigation & Whitelisting Protocol)

When implementing automated Intrusion Prevention Systems (IPS), bidirectional traffic analysis poses a critical risk of "Self-Lockout". A DNS response packet originating from the internal Gateway back to the Victim carries the malicious domain payload. Without contextual awareness, the AI will classify the Gateway as the source of the attack.

To safely deploy automated `iptables` mitigation:

1. **Define a Whitelist:** Hardcode critical infrastructure IPs (Gateway, Internal DNS).
2. **Bidirectional Blocking:** Execute both `iptables -A FORWARD -s {ip} -j DROP` (to sever external C&C connections) and `iptables -A INPUT -s {ip} -j DROP` (to sever direct internal queries).

8. Next Step

- Replace the hardcoded "Mock Dataset" with a massive real-world CSV dataset containing prominent CDN signatures to eliminate edge-case False Positives while maintaining zero-trust against tunneling payloads.